Practical:08 Date:09-10-2026

# Hyperparameter Optimization – I

**Procedure: Grid Search and Random Search for Classification and Regression**

**Part A: Classification**

1. Import the required libraries for data handling, preprocessing, model building, hyperparameter tuning, and evaluation.
2. Load the Breast Cancer dataset.
3. Separate the dataset into input variables (X) and the target variable (y).
4. Split the data into training data (80%) and testing data (20%).
5. Apply feature scaling using StandardScaler and build a pipeline with Logistic Regression.
6. Grid Search: Define a range of C values and use GridSearchCV with 5-fold cross-validation to find the best parameter.
7. Fit the Grid Search model using the training data and identify the best parameters and cross-validation accuracy.
8. Predict the test data and calculate the test accuracy.
9. Random Search: Define a range of C values using a logarithmic scale and use RandomizedSearchCV to select 10 combinations with 5-fold cross-validation.
10. Fit the Random Search model and identify the best parameters and cross-validation accuracy.
11. Predict the test data and calculate the test accuracy.

**Part B: Regression**

12. Load the Diabetes dataset.
13. Separate the dataset into input variables (X) and the target variable (y).
14. Split the data into training data (80%) and testing data (20%).
15. Apply feature scaling using StandardScaler and build a pipeline with Ridge Regression.
16. Grid Search: Define a range of alpha values and use GridSearchCV with 5-fold cross-validation to find the best parameter.
17. Fit the Grid Search model and identify the best alpha value and cross-validation Mean Absolute Error (MAE).
18. Predict the test data and calculate the test MAE.
19. Random Search: Define a range of alpha values using a logarithmic scale and use RandomizedSearchCV to select 10 combinations with 5-fold cross-validation.

20. Fit the Random Search model and identify the best alpha value and cross-validation MAE.
21. Predict the test data and calculate the test MAE.

**Code:**

```
# Grid Search and Random Search for Classification and Regression
import numpy as np
from sklearn.datasets import load_breast_cancer, load_diabetes
from sklearn.model_selection import (
    train_test_split,
    GridSearchCV,
    RandomizedSearchCV)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import accuracy_score, mean_absolute_error

# PART A: CLASSIFICATION
print("========== CLASSIFICATION ==========")
data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y)

model = Pipeline([
    ('scaler', StandardScaler()),
```

```
    ('logistic', LogisticRegression(max_iter=5000))])

# Grid Search
param_grid = {
    'logistic__C': [0.01, 0.1, 1, 10, 100]}

grid_search = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring='accuracy')

grid_search.fit(X_train, y_train)
print("\nGrid Search")
print("Best Parameters:", grid_search.best_params_)
print("Best CV Accuracy:", grid_search.best_score_)

y_pred_grid = grid_search.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred_grid))

# Random Search
param_random = {
    'logistic__C': np.logspace(-5, 2, 20)}

random_search = RandomizedSearchCV(
    model,
    param_random,
    n_iter=10,
    cv=5,
    scoring='accuracy',
```

```
    random_state=42)

random_search.fit(X_train, y_train)
print("\nRandom Search")
print("Best Parameters:", random_search.best_params_)
print("Best CV Accuracy:", random_search.best_score_)

y_pred_random = random_search.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred_random))

# PART B: REGRESSION

print("\n========== REGRESSION ==========")

data = load_diabetes()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42)

model = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge())])

# Grid Search
param_grid = {
```

```
    'ridge__alpha': [0.01, 0.1, 1, 10, 100]}

grid_search = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring='neg_mean_absolute_error')

grid_search.fit(X_train, y_train)
print("\nGrid Search")
print("Best Parameters:", grid_search.best_params_)
print("Best CV MAE:", -grid_search.best_score_)

y_pred_grid = grid_search.predict(X_test)
print("Test MAE:", mean_absolute_error(y_test, y_pred_grid))

# Random Search
param_random = {
    'ridge__alpha': np.logspace(-3, 3, 20)}

random_search = RandomizedSearchCV(
    model,
    param_random,
    n_iter=10,
    cv=5,
    scoring='neg_mean_absolute_error',
    random_state=42)

random_search.fit(X_train, y_train)
print("\nRandom Search")
```

```
print("Best Parameters:", random_search.best_params_)
print("Best CV MAE:", -random_search.best_score_)

y_pred_random = random_search.predict(X_test)
print("Test MAE:", mean_absolute_error(y_test, y_pred_random))
```

**Output:**

```
========== CLASSIFICATION ==========
Grid Search
Best Parameters: {'logistic__C': 0.1}
Best CV Accuracy: 0.9802197802197803
Test Accuracy: 0.9736842105263158

Random Search
Best Parameters: {'logistic__C': np.float64(0.11288378916846883)}
Best CV Accuracy: 0.9802197802197803
Test Accuracy: 0.9736842105263158

========== REGRESSION ==========
Grid Search
Best Parameters: {'ridge__alpha': 10}
Best CV MAE: 45.401455944145546
Test MAE: 42.85682524780061

Random Search
Best Parameters: {'ridge__alpha': np.float64(2.976351441631316)}
Best CV MAE: 45.440051928254334
Test MAE: 42.828881923799166
```